

Transformer

导论

深度学习模型的发展，本质上是模型对数据结构与信息依赖关系建模能力不断增强的过程。CNN 通过局部连接与参数共享提取空间特征，推动了图像识别的发展；RNN 通过隐藏状态实现序列信息的递归传递，使神经网络具备时间建模能力；LSTM 在 RNN 的基础上引入记忆单元与门控机制，缓解了长期依赖中的梯度消失问题；Transformer 则利用自注意力机制直接建立全局信息之间的联系，并通过并行计算显著提升了模型的训练效率。

本章将按照模型结构的演进顺序，对 CNN、RNN、LSTM 和 Transformer 的核心思想、网络结构及数学原理进行介绍，重点分析卷积运算、循环状态更新、门控记忆机制以及自注意力机制的计算过程，从而理解不同模型在局部特征提取、序列建模、长期信息记忆和全局关系建模方面的特点。

CNN → RNN → LSTM → Transformer

一、CNN：利用卷积提取局部特征

卷积神经网络 CNN 的思想是局部连接和参数共享。

假设输入图像为 $X \in \mathbb{R}^{H \times W \times C_{in}}$ ，其中 H 表示图像高度； W 表示图像宽度； C_{in} 表示输入通道数。

卷积核表示为 $K \in \mathbb{R}^{k_h \times k_w \times C_{in} \times C_{out}}$ ，卷积输出为 $Y \in \mathbb{R}^{H' \times W' \times C_{out}}$ 。

对于输出位置 (i, j) 和输出通道 c ，卷积过程为：

$$Y_{i,j,c} = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} \sum_{d=1}^{C_{in}} K_{m,n,d,c} X_{i+m,j+n,d} + b_c \quad (1)$$

再经过非线性激活函数 $Z_{i,j,c} = \sigma(Y_{i,j,c})$

常用的 ReLU 激活函数为 $ReLU(x) = \max(0, x)$

卷积核会在整张图像上滑动，因此同一组参数会作用于不同位置。

假设卷积核大小为 $k \times k$ ，输入输出通道数分别为 C_{in} 和 C_{out} ，卷积层参数数量为：

$$N_{CNN} = k^2 C_{in} C_{out} + C_{out} \quad (2)$$

参数数量与图像的空间尺寸无关，这就是参数共享。

随着卷积层不断增加，感受野逐渐扩大。若每层卷积核大小均为 k ，步长为 1，第 l 层感受野可以近似表示为 $R_l = R_{l-1} + k - 1$

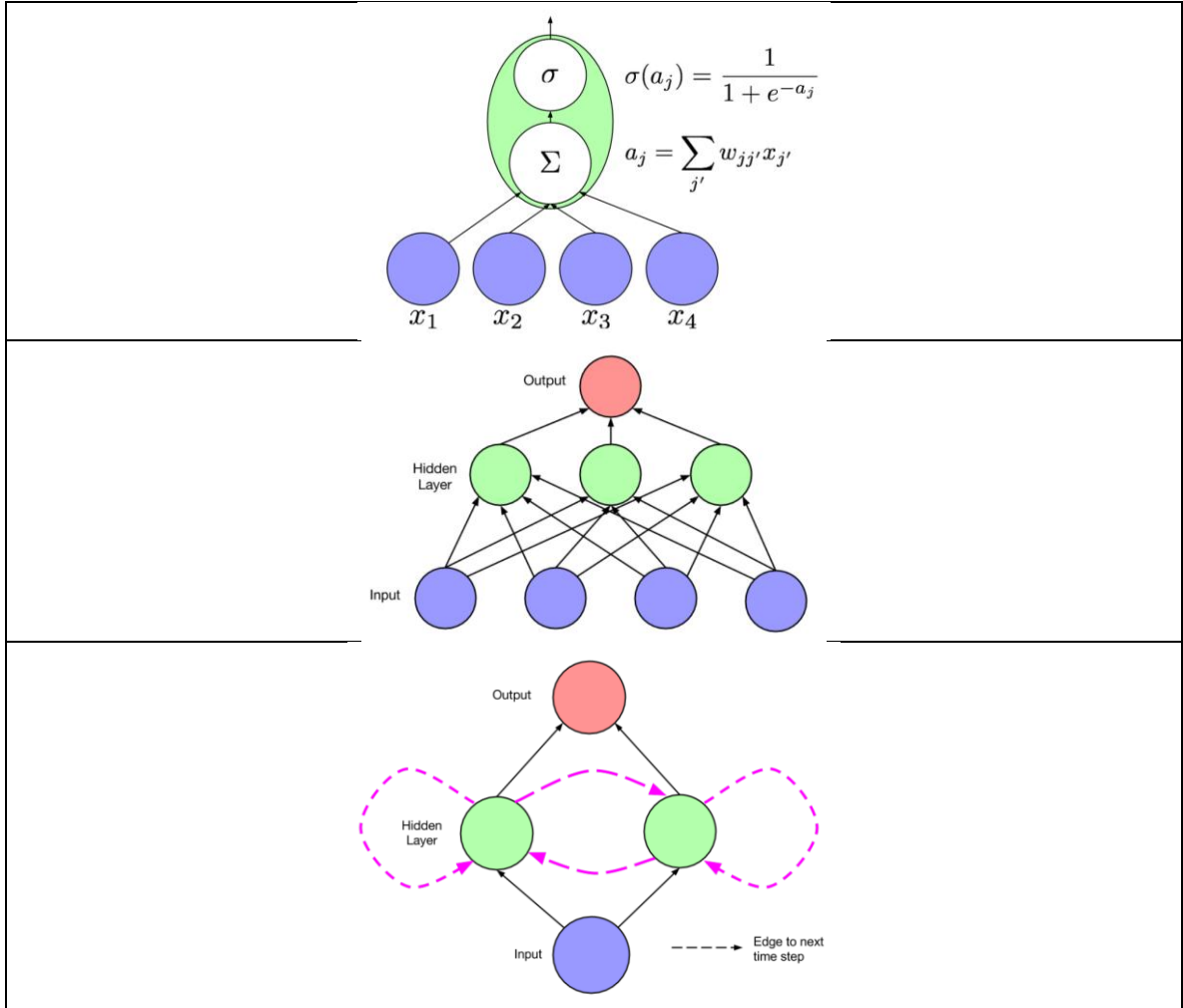
初始条件为 $R_0 = 1$ ，因此 $R_l = 1 + l(k - 1)$

CNN 擅长处理图像中的局部空间关系，但传统 CNN 本身不具备显式的时间状态，因此不适合直接描述序列中前后时刻的依赖关系。

二、RNN：利用隐藏状态传递历史信息

循环神经网络 RNN 在每一个时间步维护一个隐藏状态。

给定输入序列 $X = [x_1, x_2, \dots, x_T]$ ，当前时刻的隐藏状态由当前输入和上一时刻隐藏状态共同决定 $h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h)$ ，其中 $x_t \in \mathbb{R}^{d_x}$ ， $h_t \in \mathbb{R}^{d_h}$ ，输出可以表示为 $y_t = W_y h_t + b_y$



将隐藏状态递归展开。

$$\text{第一时刻 } h_1 = f(W_x x_1 + W_h h_0)$$

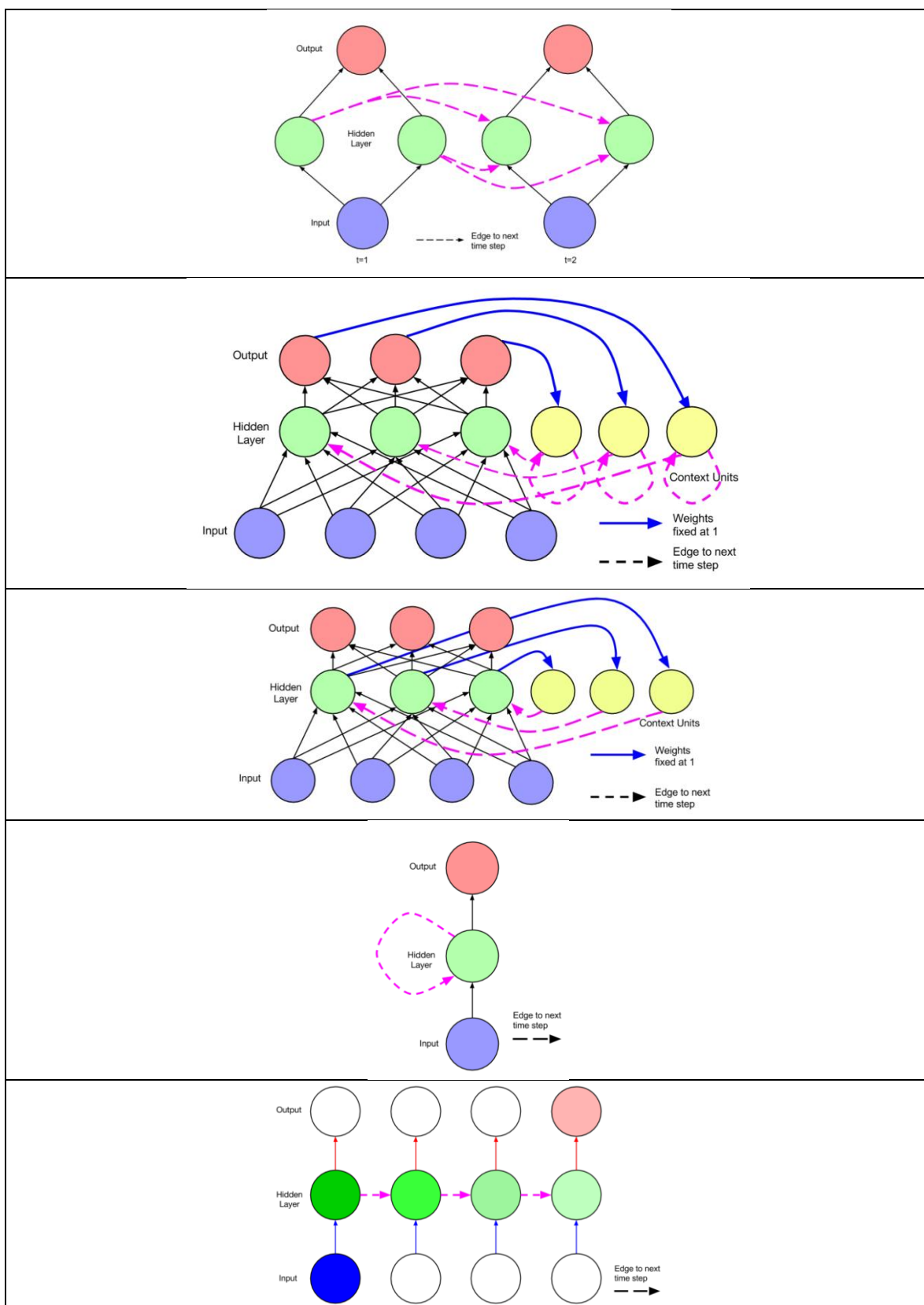
$$\text{第二时刻 } h_2 = f(W_x x_2 + W_h h_1)$$

$$\text{代入 } h_1 \text{ 可得 } h_2 = f(W_x x_2 + W_h f(W_x x_1 + W_h h_0))$$

继续递归可以得到：

$$h_t = f(x_t, f(x_{t-1}, f(\dots f(x_1)))) \quad (3)$$

因此，理论上 h_t 包含从 x_1 和 x_t 的历史信息。



假设最终损失函数为 $\mathcal{L} = \mathcal{L}(h_T)$ ，要计算较早状态 h_k 对损失的影响，需要使用链式法则：
$$\frac{\partial \mathcal{L}}{\partial h_k} = \frac{\partial \mathcal{L}}{\partial h_T} \prod_{t=k+1}^T \frac{\partial h_t}{\partial h_{t-1}}$$
，由于 $h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h)$ ，因此

$$\frac{\partial h_t}{\partial h_{t-1}} = D_t W_h, \text{ 其中 } D_t = \text{diag}(1 - h_t^2), \text{ 于是 } \frac{\partial \mathcal{L}}{\partial h_k} = \frac{\partial \mathcal{L}}{\partial h_T} \prod_{t=k+1}^T D_t W_h$$

梯度大小可以近似估计为 $\left| \frac{\partial \mathcal{L}}{\partial h_k} \right| \leq \left| \frac{\partial \mathcal{L}}{\partial h_T} \right| \prod_{t=k+1}^T |D_t W_h|$, 当 $|D_t W_h| < 1$, 随着时间距离增加 $\left| \frac{\partial \mathcal{L}}{\partial h_k} \right| \rightarrow 0$ 产生梯度消失; 当 $|D_t W_h| > 1$, 梯度可能不断增大 $\left| \frac{\partial \mathcal{L}}{\partial h_k} \right| \rightarrow \infty$, 产生梯度爆炸。

因此, 普通 RNN 很难学习距离较远的信息。

三、LSTM: 利用门控机制保存长期信息

LSTM 在 RNN 的基础上增加记忆单元 c_t

普通 RNN 只有隐藏状态 h_t

LSTM 同时维护隐藏状态和记忆状态 (h_t, c_t)

LSTM 使用三个门控制信息流动: 遗忘门; 输入门; 输出门

3.1 遗忘门

遗忘门决定上一时刻的记忆保留多少。

首先拼接当前输入和上一隐藏状态: $z_t = [h_{t-1}, x_t]$

遗忘门为 $f_t = \sigma(W_f z_t + b_f)$, Sigmoid 函数为 $\sigma(x) = \frac{1}{1 + e^{-x}}$, 因此 $0 < f_t < 1$, 当 $f_t \approx 1$ 过去的信息基本保留; 当 $f_t \approx 0$, 过去的信息基本被遗忘。

3.2 输入门

输入门决定当前信息写入记忆单元的比例 $i_t = \sigma(W_i z_t + b_i)$

生成新的候选记忆 $\tilde{c}_t = \tanh(W_c z_t + b_c)$

输入门控制候选记忆的写入量 $i_t \odot \tilde{c}_t$

3.3 更新记忆状态

新的记忆状态由旧记忆和新信息共同组成: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$

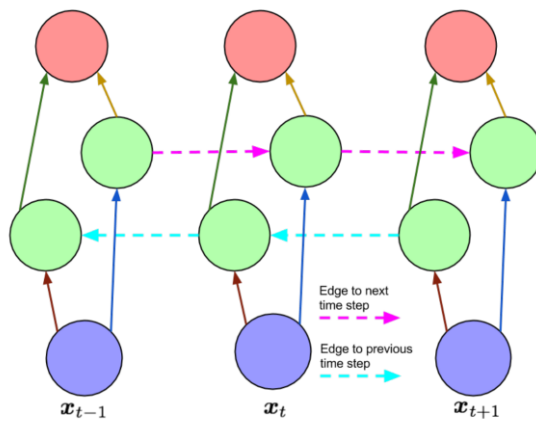
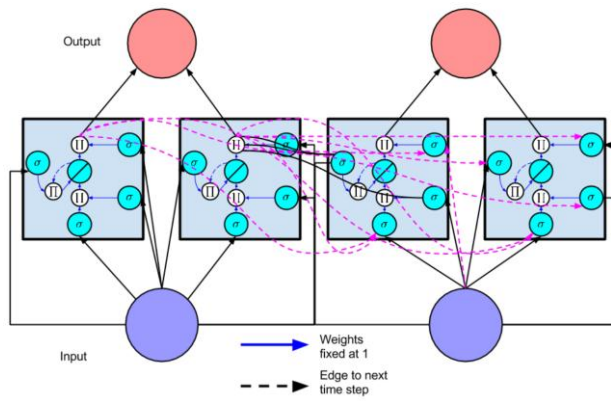
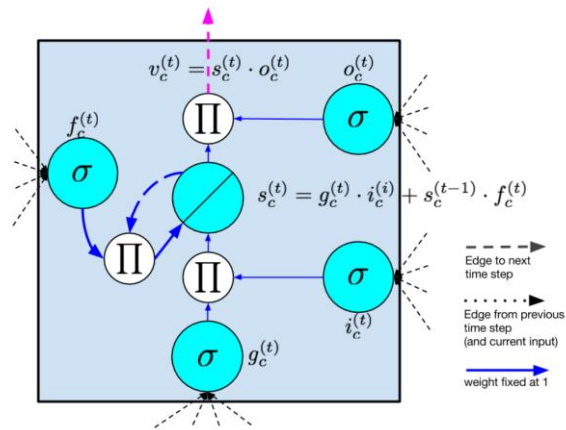
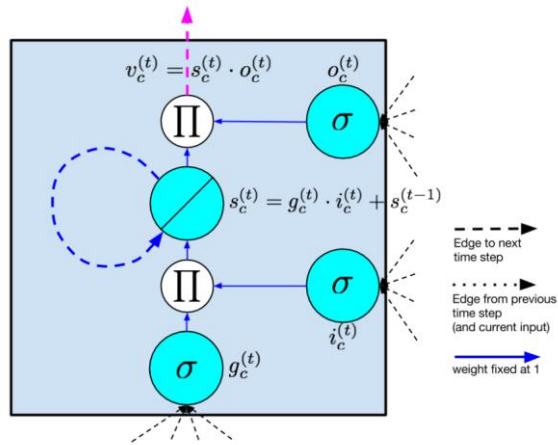
其中 $f_t \odot c_{t-1}$ 表示保留的历史信息, 而 $i_t \odot \tilde{c}_t$ 表示写入的新信息。

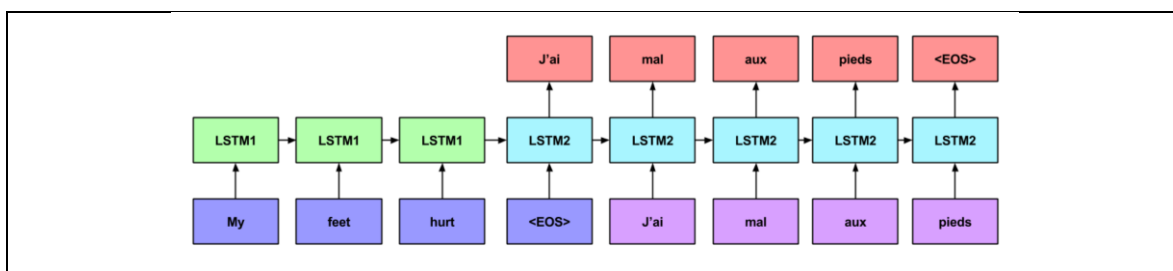
LSTM 的关键是记忆状态采用加法更新, 而不是普通 RNN 中连续的非线性矩阵变换。

3.4 输出门

输出门决定记忆单元中的哪些信息作为隐藏状态输出: $o_t = \sigma(W_o z_t + b_o)$

最终隐藏状态为 $h_t = o_t \odot \tanh(c_t)$





3.5 LSTM 为什么能够缓解梯度消失

记忆状态满足 $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ ，忽略门函数之间复杂的间接依赖，可以近似得到 $\frac{\partial c_t}{\partial c_{t-1}} \approx f_t$ ，经过多个时间步 $\frac{\partial c_T}{\partial c_k} \approx \prod_{t=k+1}^T f_t$

当遗忘门接近 1 时： $f_t \approx 1$ ，梯度可以较长时间保持 $\frac{\partial c_T}{\partial c_k} \approx 1$ ，因此，LSTM 能够学习比普通 RNN 更长距离的依赖关系。

但 LSTM 仍然需要顺序计算，计算 h_t 之前，必须先得到 h_{t-1}

计算过程为 $h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow \dots \rightarrow h_T$ ，因此难以在时间维度进行完全并行计算。

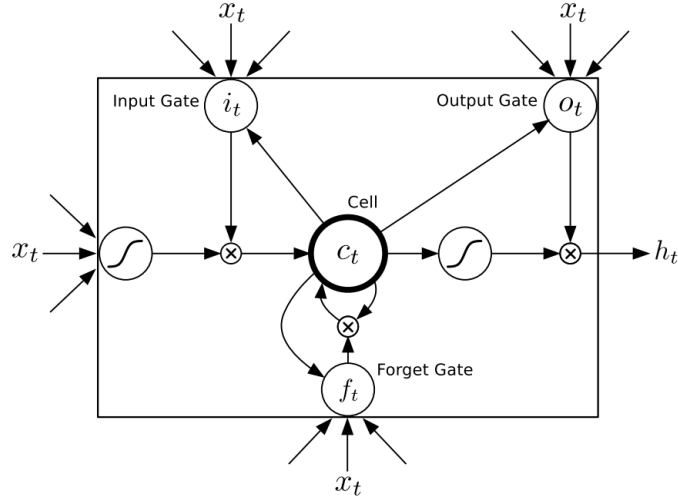


Fig. 1. Long Short-term Memory Cell

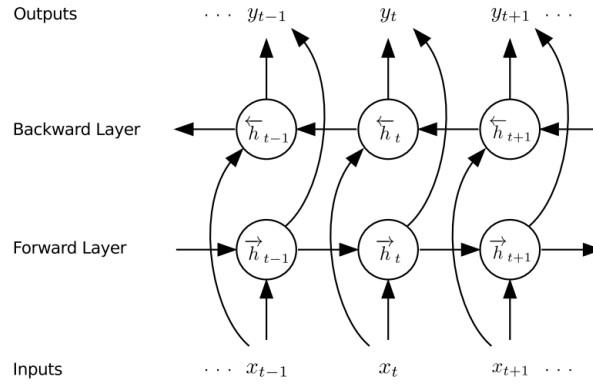


Fig. 2. Bidirectional RNN

四、Transformer：利用注意力直接建立全局关系

Transformer 去掉了 RNN 的循环结构，其思想是自注意力机制。

假设输入序列包含 N 个词。每个词被表示为 d 维向量： $X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}$ ，输入矩阵维

度为 $X \in \mathbb{R}^{N \times d}$

通过三个不同的线性变换得到查询矩阵、键矩阵和值矩阵：

$$Q = XW_Q; K = XW_K; V = XW_V$$

其中 $W_Q \in \mathbb{R}^{d \times d_k}; W_K \in \mathbb{R}^{d \times d_k}; W_V \in \mathbb{R}^{d \times d_v}$

因此 $Q \in \mathbb{R}^{N \times d_k}; K \in \mathbb{R}^{N \times d_k}; V \in \mathbb{R}^{N \times d_v}$

4.1 计算词与词之间的相关性

第 i 个词的查询向量表示为 q_i ；第 j 个词的键向量表示为 k_j

两个词之间的相关性通过点积计算 $s_{ij} = q_i^T k_j$

将所有位置同时计算，可以得到 $S = QK^T$ ，注意力分数矩阵维度为 $S \in \mathbb{R}^{N \times N}$ ，其中第 i 行第 j 列表示第 i 个词对第 j 个词的关程度。

4.2 缩放点积

假设查询向量和键向量中的每个元素均满足：

$$\mathbb{E}[q_i] = 0; \mathbb{E}[k_i] = 0; \text{Var}(q_i) = 1; \text{Var}(k_i) = 1$$

点积为 $q^T k = \sum_{i=1}^{d_k} q_i k_i$ ，其方差约为 $\text{Var}(q^T k) = d_k$ ，随着维度增加，点积数值可能变得较大。

因此除以 $\sqrt{d_k}$ ，得到缩放注意力分数 $S = \frac{QK^T}{\sqrt{d_k}}$ ，缩放后方差约为 $\text{Var}\left(\frac{q^T k}{\sqrt{d_k}}\right) \approx 1$ ，这样可以避免 Softmax 过早进入饱和区域。

4.3 计算注意力权重

对每一行进行 Softmax 归一化：

$$A = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \quad (4)$$

单个注意力权重为 $\alpha_{ij} = \frac{\exp\left(\frac{q_i^T k_j}{\sqrt{d_k}}\right)}{\sum_{m=1}^N \exp\left(\frac{q_i^T k_m}{\sqrt{d_k}}\right)}$ ，注意力权重满足 $0 < \alpha_{ij} < 1$ 并且

$$\sum_{j=1}^N \alpha_{ij} = 1$$

4.4 加权融合全局信息

第 i 个位置的输出为 $z_i = \sum_{j=1}^N \alpha_{ij} v_j$ ，矩阵形式为 $Z = AV$

代入注意力矩阵 $Z = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

完整注意力公式为：

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (5)$$

由于每个位置都能够直接关注序列中的所有位置，因此 Transformer 可以直接建立全局依赖。

五、多头注意力

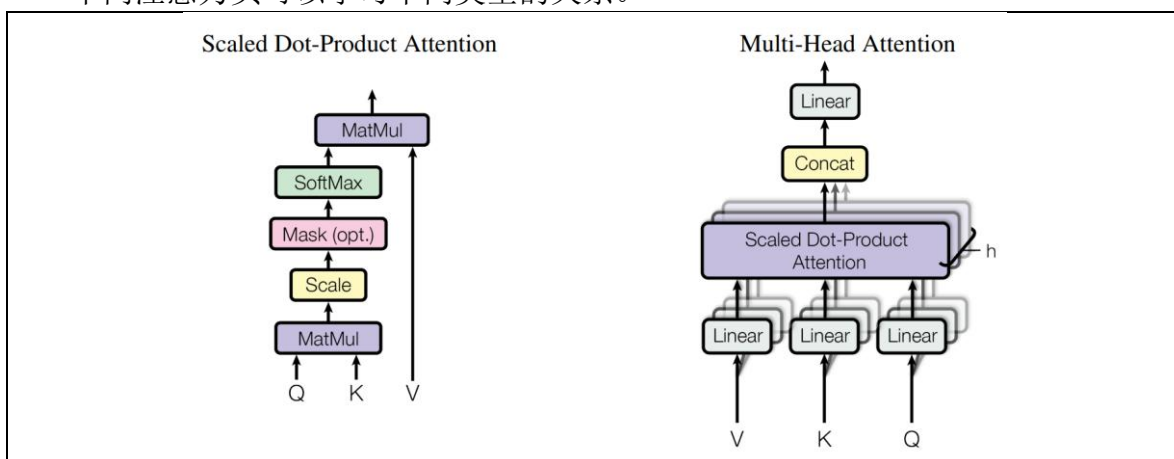
单个注意力头只能在一个特征空间中计算关系，Transformer 使用多个注意力头：

$$head_i = Attention(XW_i^Q, XW_i^K, XW_i^V) \quad (6)$$

将多个注意力头拼接： $H = Concat(head_1, head_2, \dots, head_h)$

再进行线性映射： $MultiHead(X) = HW^O$

不同注意力头可以学习不同类型的关系。



六、Transformer 的完整结构

自注意力输出首先经过残差连接和归一化：

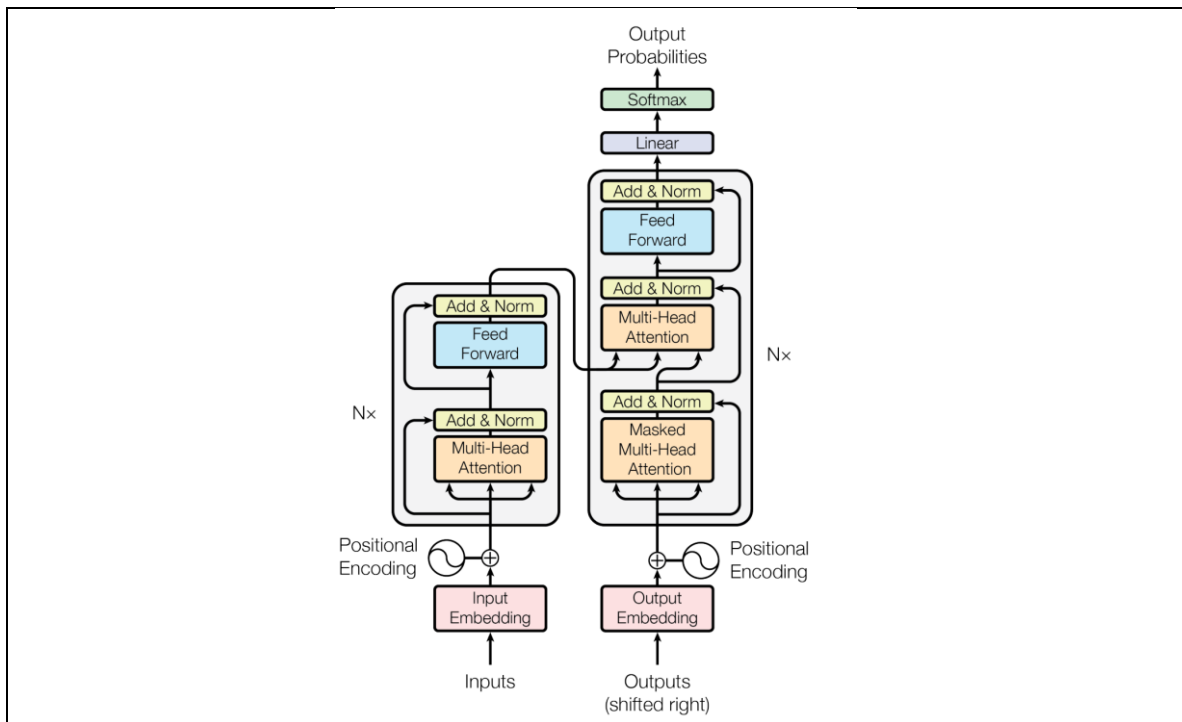
$$X' = LayerNorm(X + MultiHead(X)) \quad (7)$$

然后经过前馈神经网络：

$$FFN(X') = \sigma(X'W_1 + b_1)W_2 + b_2 \quad (8)$$

再次进行残差连接：

$$Y = LayerNorm(X' + FFN(X')) \quad (9)$$



Transformer 相比 RNN 和 LSTM 的变化，不只是加入注意力机制，而是彻底取消循环依赖，使所有位置能够并行计算，并通过自注意力直接建立任意位置之间的联系。